

SQL Basic Join Functions

1 · Basic Joins — Complete Reference Guide (Banking Project)

Table of Contents

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL JOIN

1. INNER JOIN

→ **Task 1: Retrieve a list of all customers along with their account details, showing only customers who currently own at least one account.**

```
SELECT
    c.customer_id, c.first_name, c.last_name, c.customer_segment,
    a.account_id, a.account_type, a.account_balance
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id;
```

→ **Task 2: Build a successful-transaction activity report showing the customer's name and segment together with the account type for every successful transaction.**

```
SELECT
    t.transaction_id, t.transaction_date, t.transaction_amount,
    t.transaction_type, a.account_type,
    c.first_name, c.last_name, c.customer_segment
FROM Transactions t
INNER JOIN Accounts a
    ON t.account_id = a.account_id
INNER JOIN Customers c
    ON a.customer_id = c.customer_id
WHERE t.transaction_status = 'Success';
```

→ **Task 3: Identify every account whose current balance has fallen below its required minimum balance, along with the owning customer's name and segment, for the penalty-fee report.**

```
SELECT
    c.customer_id, c.first_name, c.last_name, c.customer_segment,
    a.account_id, a.account_type, a.account_balance, a.min_balance_required
FROM Accounts a
INNER JOIN Customers c
    ON a.customer_id = c.customer_id
WHERE a.account_balance < a.min_balance_required;
```

2. LEFT JOIN

→ **Task 4: List every customer together with their accounts, including customers who do not yet own any account, so the sales team can target them for account onboarding.**

```
SELECT
    c.customer_id, c.first_name, c.last_name, c.customer_segment,
    a.account_id, a.account_type, a.account_balance
FROM Customers c
LEFT JOIN Accounts a
    ON c.customer_id = a.customer_id;
```

→ **Task 5: List every account together with the date of its most recent transaction, including accounts that have never had a transaction, to flag dormant accounts for the retention team.**

```
SELECT
    a.account_id, a.account_type, a.account_status,
    MAX(t.transaction_date) AS last_transaction_date
FROM Accounts a
LEFT JOIN Transactions t
    ON a.account_id = t.account_id
GROUP BY a.account_id, a.account_type, a.account_status;
```

→ **Task 6: Calculate each customer's total balance across all of their accounts, including customers with zero accounts shown as 0, for the total-relationship-value report used in customer segmentation.**

```
SELECT
    c.customer_id, c.first_name, c.last_name, c.customer_segment,
    ISNULL(SUM(a.account_balance), 0) AS total_balance
FROM Customers c
LEFT JOIN Accounts a
    ON c.customer_id = a.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.customer_segment;
```

3. RIGHT JOIN

→ **Task 7: Using RIGHT JOIN, retrieve every account along with its owning customer's details, written from the Customers side so learners can see that RIGHT JOIN produces the same result as swapping table order with LEFT JOIN.**

```
SELECT
    c.customer_id, c.first_name, c.last_name,
    a.account_id, a.account_type, a.account_balance
FROM Customers c
RIGHT JOIN Accounts a
    ON c.customer_id = a.customer_id;
```

→ **Task 8: Using RIGHT JOIN**, retrieve every account along with all of its transactions (if any), written from the Transactions side, so every account appears even when it has no transactions.

```
SELECT
    t.transaction_id, t.transaction_type, t.transaction_amount,
    a.account_id, a.account_type, a.account_status
FROM Transactions t
RIGHT JOIN Accounts a
    ON t.account_id = a.account_id;
```

→ **Task 9: Using RIGHT JOIN combined with a NULL check**, identify accounts that have never had a single transaction, for the retention team's zero-activity account list.

```
SELECT
    a.account_id, a.account_type, a.account_status, a.open_date
FROM Transactions t
RIGHT JOIN Accounts a
    ON t.account_id = a.account_id
WHERE t.transaction_id IS NULL;
```

4. FULL JOIN

→ **Task 10: Produce a full reconciliation report between Customers and Accounts**, returning every customer (even without an account) and every account in one result set.

```
SELECT
    c.customer_id, c.first_name, c.last_name,
    a.account_id, a.account_type
FROM Customers c
FULL JOIN Accounts a
    ON c.customer_id = a.customer_id;
```

→ **Task 11: Produce a full reconciliation report between Accounts and Transactions**, returning every account whether or not it has transactions, and every transaction in the same result set.

```
SELECT
    a.account_id, a.account_type,
    t.transaction_id, t.transaction_type, t.transaction_amount
FROM Accounts a
FULL JOIN Transactions t
    ON a.account_id = t.account_id;
```

→ **Task 12: Using FULL JOIN with a CASE expression, classify every Customer-Account pairing as 'Matched', 'Customer Only - No Account', or 'Account Only - No Customer' for the data-quality team's reconciliation dashboard.**

```
SELECT
  c.customer_id, c.first_name, c.last_name,
  a.account_id, a.account_type,
  CASE
    WHEN c.customer_id IS NOT NULL AND a.account_id IS NOT NULL
      THEN 'Matched'
    WHEN c.customer_id IS NOT NULL AND a.account_id IS NULL
      THEN 'Customer Only - No Account'
    ELSE 'Account Only - No Customer'
  END AS reconciliation_status
FROM Customers c
FULL JOIN Accounts a
  ON c.customer_id = a.customer_id;
```